



C++ Alliance Fellowship @ Compiler Research

Contributors: Vipul Cariappa, Ayokunle Amodu, Aditya Medhane, Muhammad Bassiouni, Peiqi Li, Ezzeldin Ibrahim, Sahil Patidar

Mentors: Vassil Vassilev, Aaron Jomy

Program Administration: Martin Vassilev



About the Fellowship

*Build the parts of **Clang** that matter next.*

A remote, mentor-driven fellowship run by **Compiler Research** and sponsored by the **C++ Alliance**. It takes contributors from a first patch to production-quality code in the main branch of **llvm-project** — work that ships to everyone who uses **Clang**.

- **Upstream first.** Projects land as reviewed commits in **llvm-project** itself: no forks, no side branches.
- **Mentor-driven.** Week-by-week guidance from experienced LLVM maintainers and reviewers.
- **Global and remote.** Open worldwide, flexible participation, first-time contributors welcome.

Focus areas

C++20/23/26 language support; compile-time and memory performance; diagnostics and developer experience; tooling — **clang-tidy**, the static analyzer, **clang-repl**.

The 2026 cohort

7 fellows	5 countries, 4 continents	7 project tracks	6 mo full-time equivalent
---------------------	-------------------------------------	----------------------------	-------------------------------------

A year in the programme

Applications	Mar 15 – May 1
Community bonding	May 4 – 24
Coding I	May 25 – Jul 26
Midterm evaluation	Jul 27 – Aug 2
Coding II & final	Aug 3 – Oct 25

Location-fair stipends, following the GSoC purchasing-power tables, paid in two halves at the midterm and final evaluations.

Vipul Cariappa

Project

On-Demand Parsing in Clang

Mentors: Vassil Vassilev, Aaron Jomy.

Summary

A typical C++ translation unit includes many header files, but only a small subset of their declarations and definitions is actually used. Despite this, compilers eagerly parse all included code, causing unnecessary computational overhead.

This project aims to design and implement a mechanism in the Clang compiler to parse function and class definitions only when used.

Progress So Far

- Added on-demand parsing for internal-linkage functions & methods behind a new flag **-fparse-functions-ondemand**, while making name lookup location-aware to preserve correct semantics.
- Applying on-demand parsing in incremental mode (clang-repl), where the internal-linkage restriction can be lifted entirely, offering the greatest potential gains.

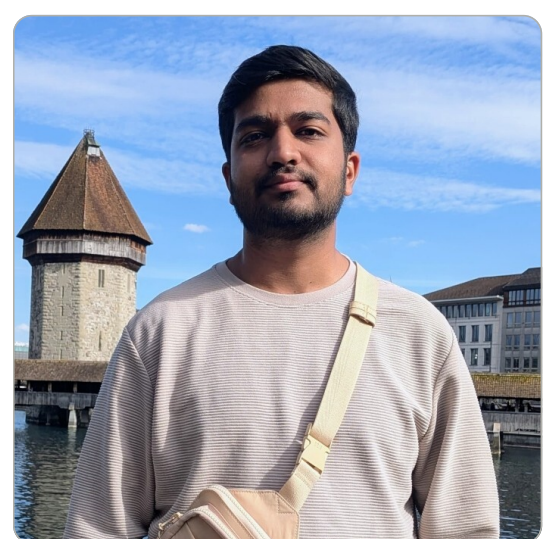
Background

Summary: Open-source contributor and maintainer across projects involving compilers and developer tools. Currently working as a research fellow at FPLaunchpad, IIT-M.

Education: Bachelor of Technology in Computer Science and Engineering, Ramaiah University of Applied Sciences, India, 2021 – 2026

Country: 🇨🇭 Switzerland

Email: vipulcariappa@gmail.com



Ayokunle Amodu

Project

Optimize Usage of Source Locations in Clang Modules.

Mentors: Vassil Vassilev, Aaron Jomy.

Summary

Clang gives every token a source location inside fixed 32-bit address space, and large modular builds can exhaust it and crash. Much of the waste is duplication: when many modules include the same header, each loads its own copy of that file's source locations. This work deduplicates them in the module loader, so a file an earlier module already loaded is reused instead of recopied.

Progress So Far

- Deduplication works end to end and is up for review on llvm-project, with the core mechanism in place and moving toward a merge.
- Compile time is negligible; peak memory rises a little from the small per-module mapping tables the remapping adds, which is next to bring down.
- Tested downstream at Apple, CERN, Google and Alibaba against their own module-heavy codebases, currently working through crashes from a large real-world build.

Background

Summary: Compiler engineer and LLVM Project member working across Clang, MLIR and ClangIR. I love learning how compilers interact with hardware.

Education: Bachelor of Computing Science, University of Alberta, Canada, 2022 – 2026

Country: 🇨🇦 Canada

Email: ayokunle@gmail.com



<https://compiler-research.org/team/ayokunleamodu>



<https://www.linkedin.com/in/ayokunle-amodu>



<https://github.com/ayokunle321>

Aditya Medhane

Project

Enhance Clang Diagnostics.

Mentors: Vassil Vassilev, Aaron Jomy.

Summary

Close gaps in Clang's warnings: fix diagnostics that miss real bugs, flag undefined behavior the C standard calls out, and turn useful opt-in warnings on by default. 26 upstream PRs in **llvm-project** so far.

Progress So Far

- Fixed **-Wtautological-pointer-compare** to fire through function references (**merged**).
- Added a diagnostic for conflicting linkage, UB per WG14 N3410 (targeting **Clang 24**).
- Enabled **-Wunused-template** under **-Wall** after 20 monorepo-wide cleanup PRs (**live on main**); already driving fixes in Bitcoin Core and HarfBuzz.
- Next: moving type-safety checks from **clang-tidy** into Sema.

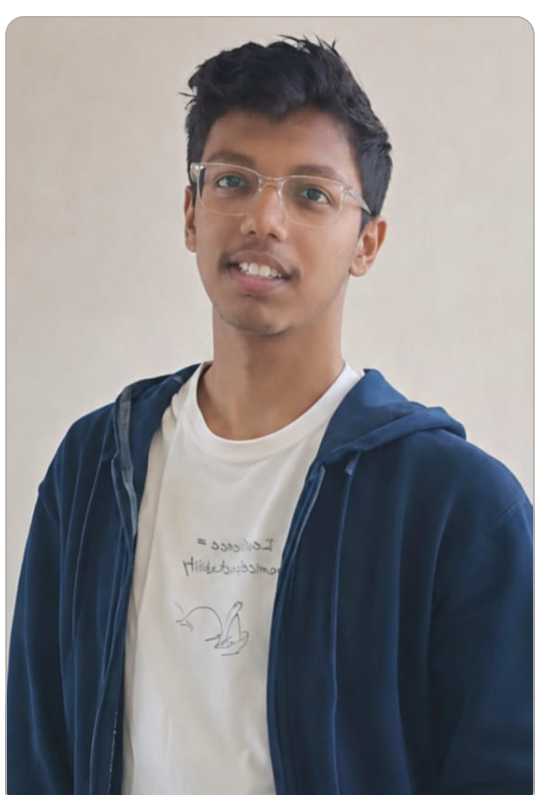
Background

Summary: Open-source contributor across compilers and iOS; WWDC 2025 Swift Student Challenge winner.

Education: Bachelor of Technology in Computer Science, ABV-IIITM Gwalior, India, 2023 – 2027

Country: 🇮🇳 India

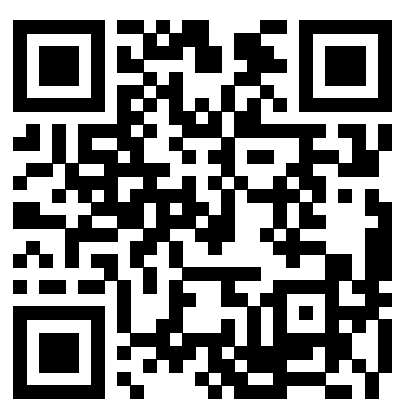
Email: adityamedhane.dev@gmail.com



<https://compiler-research.org/team/AdityaMedhane>



<https://www.linkedin.com/in/aditya-medhane>



<https://github.com/flaahi729>

Muhammad Bassiouni

Project

Implement missing C++26 features in Clang.

Mentors: Vassil Vassilev, Aaron Jomy.

Summary

Bring language features to **Clang** compiler, starting with *if declarations* (C2y paper N3356) and continuing with class template argument deduction (CTAD) for alias templates (P1814R0), each with diagnostics and a conformance test suite.

Progress So Far

- *If declarations* (N3356) implemented and **merged** into **Clang 24.0.0git**, reusing and refactoring the existing C++ condition parser for C mode.
- Backported *If declarations* to every C mode down to **C89** as a compatible extension, with new errors, warnings, and compatibility diagnostics for the C2y grammar pitfalls.
- Added **__extension__**, **C23** attribute declarations, and **static_assert** declaration in conditions — the first of these was not previously available in C++.
- Motivated extending **Clang's DiagCompat()** compatibility-diagnostics infrastructure to C language modes, now available to all C diagnostics.
- 200+ line conformance test suite, plus release notes, *Language Extensions* docs, and the public C conformance page updated; feature-complete and ahead of current implementation in GCC.
- Currently working on CTAD for alias templates (P1814R0), toward full C++20 conformance.

Background

Summary: Software Engineer and **Open Source Maintainer** of the **LLVM C Library**, with 2.9 years of experience in compilers, toolchain technologies, and backend development. Interested in working on cutting-edge, scalable services technology with global impact.

Education: Bachelor of Artificial Intelligence in Intelligent Systems, Menoufia University, Egypt, 2021 – 2025

Country: 🇪🇬 Egypt

Email: muhammad.m.bassiouni@gmail.com



<https://compiler-research.org/team/MuhammadBassiouni>



<https://www.linkedin.com/in/bassiounix>



<https://github.com/bassiounix>

Acknowledgements

This programme is funded by the **C++ Alliance** and run by the **Compiler Research** group. Everything on this poster reached users through the LLVM Project's own public review process: the work is the fellows', the bar it had to clear is the community's.

Thanks to

- **Vassil Vassilev** and **Aaron Jomy**, for mentoring every track on this poster week by week, from a first patch to a merged commit.
- **Martin Vassilev**, for running the programme — selection, evaluations and stipends — so the fellows could stay on the code.
- The **LLVM** reviewers and code owners who read these patches across time zones, and the downstream teams who tested them against their own builds.
- **CppCon**, for the room and the conversation in front of this poster.

Partners



The C++ Alliance funds this fellowship outright: no fees, no equity, no strings on where the code lands.

Peiqi Li

Project

Process High-Impact clang-tidy and Clang Static Analyzer Requests

Mentors: Vassil Vassilev, Aaron Jomy.

Summary

Resolve high-impact issues in **clang-tidy** and the Clang Static Analyzer, focusing on false positives, false negatives, unsafe fix-it hints and requests for new checks. This work also bridges some core **clang-tidy** capabilities directly into **clangd** for language server users.

Progress So Far

- **clangd Architectural Integrations:** Decoupled **mergeFixits()** into **clang::edit** to prevent syntax errors (#208682), enabled query-based custom checks (#216336), and added native **.clang-tidy** line-filtering (#202575).
- **High-Impact Bug Fixes:** Fixed **use-after-move** AST unwrapping (#199905), **redundant-expression** type-alias false positives (#198085), and default **IgnoreExternC** header protections (#199189).

Background

Summary: Open-source contributor interested in compilers. Currently a C++ Alliance research fellow working on LLVM tooling.

Education: Bachelor of Mathematics and Artificial Intelligence, Shanghai Jiao Tong University, China, 2024 – 2028

Country: 🇨🇳 China

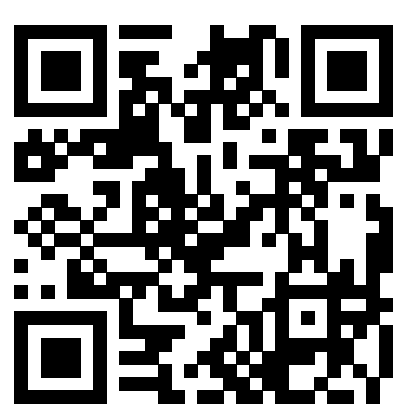
Email: voyager.lpq@gmail.com



<https://compiler-research.org/team/PeiqiLi>



<https://www.linkedin.com/in/pq-li>



<https://github.com/voyager-jhk>

Ezzeldin Ibrahim

Project

Improve Clang Performance

Mentors: Vassil Vassilev, Aaron Jomy.

Summary

Clang's compile time and peak memory drift upwards as the language grows: new features, added diagnostics and evolving usage patterns leave small inefficiencies behind that no single patch is blamed for, but that accumulate release over release into real cost for large C++ projects. This project hunts those hidden regressions systematically by benchmarking real-world C/C++ codebases across Clang 18 to trunk, bisecting each slowdown to its first-bad commit, and upstreaming fixes.

Progress So Far

- Upstreamed the fixable ones: 6 patches merged so far (typo-correction gating, -1.2% on Boost.MPL; cached analysis-based warning policy), $\sim 1.25\%$ total frontend win.

Background

Summary: Open-source contributor working on compilers and developer tooling. LLVM committer, contributing to libc and Clang. Currently a C++ Alliance research fellow.

Education: Bachelor of Computer Science, Zagazig University, Egypt, 2022 – 2026

Country: 🇪🇬 Egypt

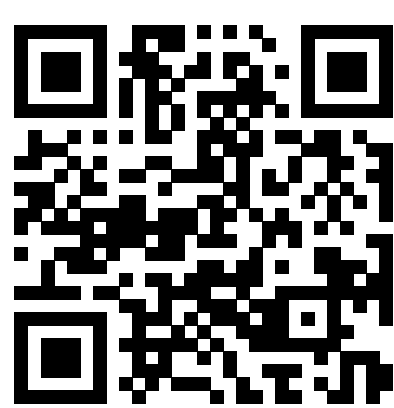
Email: ezzibrahimx@gmail.com



<https://compiler-research.org/team/EzzeldinIbrahim>



<https://www.linkedin.com/in/ezz-eldean>



<https://github.com/AnnMhraj>

Sahil Patidar

Project

Consistent Error Recovery Infrastructure

Mentors: Vassil Vassilev, Aaron Jomy.

Summary

Interactive C++ compilation keeps compiler state alive across multiple inputs, unlike traditional batch compilation. When an input fails, changes made to the shared compiler state can remain behind and affect subsequent inputs, causing incorrect diagnostics, redefinition errors, or unstable behavior.

This project aims to improve Clang's error recovery by ensuring that failed inputs can be completely reverted to the last valid compiler state, while preserving all previously successful inputs and avoiding the need to restart or rebuild the compiler.

Progress So Far

- Investigated the memory-rewind prototype through testing, studying how failed PTUs leave invalid mutations in Clang's shared AST and semantic state, and how memory rewind affects compiler state. Collected cases of dangling reference crashes and incorrect diagnostics caused by failed inputs.
- Designed PTU-aware state tracking using **ASTMutationListener**, based on the observed failure cases and state mutations across incremental inputs.
- Implementing the concrete tracking and rollback system based on the gathered findings and design.

Background

Summary: Open-source contributor to LLVM, working across projects involving ORC-JIT, clang-repl, and other parts of the LLVM project.

Education: Bachelor of Technology in Computer Science and Engineering, Vindhya Institute of Technology and Science Indore, India, 2019 – 2023

Country: 🇮🇳 India

Email: sahilpatidar60@gmail.com



<https://compiler-research.org/team/SahilPatidar>



<https://www.linkedin.com/in/sahil-patidar-854441200>



<https://github.com/SahilPatidar>

Join the 2027 cohort

Applications open March 15 – May 1, 2027

Programme & projects



<https://compiler-research.org/careers>

Ask us anything on Discord



<https://discord.gg/VKv3ne2cVK>

How to apply

1. Pick a project from the open list.
2. Discuss your plan on the Compiler Research Discord.
3. Email your proposal to **vvasilev@cern.ch**.

What a fellow gets

- Week-by-week mentorship from LLVM maintainers.
- A location-fair stipend, paid in two halves.
- Commits, RFCs and docs under your own name.

Sponsor or partner

Fund a fellow, or bring a project to the programme: **vvasilev@cern.ch**.

Part of the Compiler Research initiative · funded by the C++ Alliance · **compiler-research.org**



compiler-research.org

This poster is created with L^AT_EX by Muhammad Bassiouni.

SPONSORED BY C++ ALLIANCE